EEC 201 Computer Assignment 2

Seo Bin Han

April 3, 2026

Code for Assignment (Including Comments)

```
%% EEC 201 Computer Assignment 2
% Seo Bin Han
% Overall Explanation:
% I implemented a three-stage multirate sampling rate converter using
% the rational factorization (10/7)*(8/7)*(4/3) = 320/147  24000/11025.
% Each stage consisted of interpolation, low-pass filtering, and decimation.
% The low-pass filters were designed using the Kaiser window method exactly
% as specified in the handout.

clear; close all; clc;

%% (1) Design specifications
% I specified the desired stopband attenuation in dB and converted
% it into a linear ripple constraint for both passband and stopband.

A_dB = 30;                          % Stopband attenuation in dB
delta = 10^(-A_dB/20);              % Linear ripple magnitude
dp = delta;                         % Passband ripple
ds = delta;                         % Stopband ripple

use_fftfilt = true;                 % FFT-based convolution for efficiency
inFile  = 'Toreador.wav';
outFile = 'output.wav';
Fs_out  = 24000;

%% (2) Input audio
% I read Toreador (amazing song)'s inpute file and
% verified that the sampling rate matched the expected 11.025 kHz.
% I also handled mono and stereo cases uniformly.

[x, Fs] = audioread(inFile);
```

```
if Fs ~= 11025
    warning('Expected Fs = 11025 Hz, but received Fs = %g Hz.', Fs);
end

if size(x,2) == 1
    x = [x, x];
    wasMono = true;
else
    wasMono = false;
end

%% (3) Multistage factors
% I decomposed the overall interpolation and decimation ratio into three stages
% to reduce the required filter order and computational cost.

L = [10 8 4];                        % Interpolation factors
M = [7  7 3];                        % Decimation factors

%% (4) Kaiser Window
% For each stage, I calculated the allowable passband edge as:
%     wp = min(pi/Li, pi/Mi)
% I then defined the stopband edge as ws = 1.2*wp and the cutoff frequency
% as wc = 1.1*wp, per the project specification.
% I used kaiserord to estimate the FIR filter order and beta parameter,
% enforced an even order for linear-phase Type-I FIR filters,
% and scaled the passband gain to Li to preserve signal amplitude
% after interpolation.

h = cell(1,3);
info = struct([]);

for i = 1:3
    Li = L(i);
    Mi = M(i);

    wp = min(pi/Li, pi/Mi);
    ws = 1.2 * wp;
    wc = 1.1 * wp;

    wp_n = wp/pi;
    ws_n = ws/pi;
    wc_n = wc/pi;

    % I estimated the required FIR order and Kaiser beta parameter
    [n_est, ~, beta, ~] = kaiserord([wp_n ws_n],[1 0],[dp ds]);
```

```
    % I enforced an even-order FIR filter to guarantee linear phase
    n = n_est;
    if mod(n,2) ~= 0
        n = n + 1;
    end

    % I designed the low-pass FIR filter using fir1 with a Kaiser window
    w = kaiser(n+1, beta);
    b = fir1(n, wc_n, 'low', w);   % scaled to ~1 at DC
    b = Li * b;

    h{i} = b;

    % I stored the design parameters for later reporting and verification
    info(i).stage = i;
    info(i).Li = Li;
    info(i).Mi = Mi;
    info(i).wp = wp;
    info(i).ws = ws;
    info(i).wc = wc;
    info(i).n  = n;
    info(i).beta = beta;
end

%% (5)Multistage processing
% I processed the signal sequentially through each interpolation/filter/decimation
% stage. At each stage, the signal was first upsampled by Li, filtered to remove
% spectral images, and then downsampled by Mi to prevent aliasing.

y = x;

for i = 1:3
    Li = L(i);
    Mi = M(i);
    bi = h{i};

    % I upsampled the signal by inserting Li-1 zeros between samples
    y_up = upsample(y, Li);

    % I applied the low-pass FIR filter independently to each channel
    y_f = zeros(size(y_up));
    for ch = 1:size(y_up,2)
        if use_fftfilt
            y_f(:,ch) = fftfilt(bi, y_up(:,ch));
        else
            y_f(:,ch) = conv(y_up(:,ch), bi, 'same');
```

```
        end
    end

    % I downsampled the filtered signal by retaining every Mi-th sample
    y = downsample(y_f, Mi);
end

%% (6) Output formatting
% If the original input was mono, I removed the duplicate channel.
% I then normalized the output signal to prevent clipping and wrote
% the result to a WAV file at the target sampling frequency.

if wasMono
    y = y(:,1);
end

peak = max(abs(y(:)));
if peak > 0.999
    y = 0.999 * y / peak;
end

audiowrite(outFile, y, Fs_out);

%% (for me) Verification
dur_in  = size(x,1) / Fs;
dur_out = size(y,1) / Fs_out;

fprintf('Input Fs: %g Hz\n', Fs);
fprintf('Output Fs: %g Hz\n', Fs_out);
fprintf('Input duration : %.6f s\n', dur_in);
fprintf('Output duration: %.6f s\n', dur_out);
fprintf('Duration error : %.6e s\n', dur_out - dur_in);
ratio_emp = (length(y)/Fs_out) / (length(x)/Fs);
fprintf("Empirical time ratio = %.12f\n", ratio_emp); % must be near 1


for i = 1:3
    fprintf('Stage %d: L=%d, M=%d, FIR order=%d, beta=%.4f\n', ...
        i, info(i).Li, info(i).Mi, info(i).n, info(i).beta);
end
```

Verification

MATLAB outputted the following:

```
Input Fs: 11025 Hz
Output Fs: 24000 Hz
Input duration : 154.749388 s
Output duration: 154.749458 s
Duration error : 7.057823e-05 s
Empirical time ratio = 1.000000456081
Stage 1: L=10, M=7, FIR order=154, beta=2.1166
Stage 2: L=8, M=7, FIR order=124, beta=2.1166
Stage 3: L=4, M=3, FIR order=62, beta=2.1166
```

The empirical time ratio of 1.000000456 confirmed that the multistage rational sample-rate conversion accurately preserved signal duration, with the small mismatch arising from integer rounding of the final sample count (my assumption). Furthermore, the decreasing FIR orders across stages indicate that the multistage design successfully relaxed transition-band constraints at later stages, which achieved the target 30 dB attenuation.

The addition of a GUI

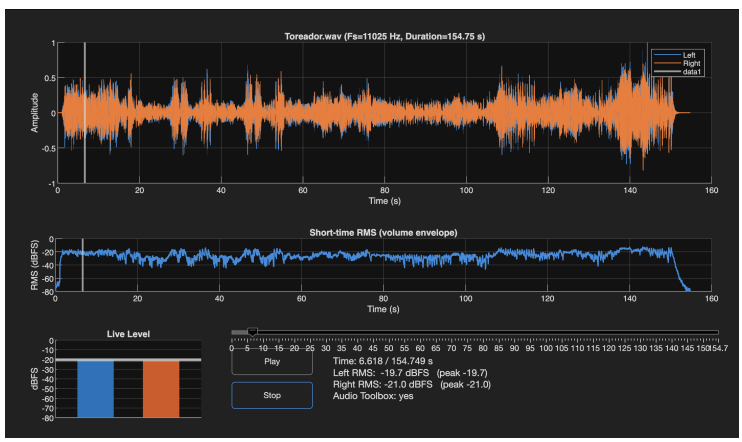

Figure 1: Image of GUI

I also implemented an interactive visualization tool for the Toreador .wav file that displayed "how loud the music is" over time while the audio played. The GUI plotted the full-track waveform (left and right channels) so I could see the raw amplitude structure across the entire recording, and it overlaid a moving playhead so the current playback position was always visible. In addition, it computed a short-time RMS envelope using a sliding window and expressed it in dBFS, which provided a compact "volume curve" that tracked perceived

loudness changes better than the raw waveform. During playback, the tool continuously measured the local RMS level around the current time and displayed it as live vertical level meters for each channel, including a peak-hold indicator so transient peaks were easy to observe.

```
function wav_meter_gui(inFile)
% Seo Bin Han
% wav_meter_gui(inFile)
% Live GUI: waveform + RMS envelope + real-time meters for a WAV file.
%
% Usage:
%   wav_meter_gui("Toreador.wav")
%   wav_meter_gui

if nargin < 1 || isempty(inFile)
    [f,p] = uigetfile({'*.wav','WAV files (*.wav)'}, 'Select a WAV file');
    if isequal(f,0), return; end
    inFile = fullfile(p,f);
end

[x, Fs] = audioread(inFile);
if size(x,2) == 1
    x = [x x];
end

x = x / max(1e-9, max(abs(x(:))));
N = size(x,1);
T = (0:N-1)/Fs;
dur = N/Fs;

winDur = 0.050;  hopDur = 0.010;
win = max(32, round(winDur*Fs));
hop = max(16, round(hopDur*Fs));

nFrames = floor((N-win)/hop) + 1;
tEnv = ((0:nFrames-1)*hop + win/2)/Fs;

rmsL = zeros(nFrames,1);
rmsR = zeros(nFrames,1);
for k = 1:nFrames
    i0 = (k-1)*hop + 1;
    seg = x(i0:i0+win-1,:);
    rmsL(k) = sqrt(mean(seg(:,1).^2));
    rmsR(k) = sqrt(mean(seg(:,2).^2));
end
```

```
rmsM = 0.5*(rmsL + rmsR);
rmsM_dB = 20*log10(rmsM + 1e-12);

player = audioplayer(x, Fs);
hasAT = true;


fig = uifigure('Name','WAV Volume Meter', 'Position',[100 100 1100 650]);

ax1 = uiaxes(fig, 'Position',[40 360 1020 260]);
plot(ax1, T, x(:,1), 'LineWidth', 1); hold(ax1,'on');
plot(ax1, T, x(:,2), 'LineWidth', 1);
grid(ax1,'on');
xlabel(ax1,'Time (s)'); ylabel(ax1,'Amplitude');
title(ax1, sprintf('%s (Fs=%g Hz, Duration=%.2f s)', stripPath(inFile), Fs, dur));
legend(ax1, {'Left','Right'}, 'Location','northeast');

ph1 = xline(ax1, 0, 'LineWidth', 2);

ax2 = uiaxes(fig, 'Position',[40 200 1020 130]);
plot(ax2, tEnv, rmsM_dB, 'LineWidth', 1.4);
grid(ax2,'on');
xlabel(ax2,'Time (s)'); ylabel(ax2,'RMS (dBFS)');
title(ax2, 'Short-time RMS (volume envelope)');
ylim(ax2, [-80 0]);
ph2 = xline(ax2, 0, 'LineWidth', 2);

% Meter axes
axM = uiaxes(fig, 'Position',[40 40 260 140]);
axis(axM, [0 1 -80 0]); grid(axM,'on');
axM.XTick = [];
axM.YTick = -80:10:0;
ylabel(axM,'dBFS');
title(axM,'Live Level');
hold(axM,'on');

barL = rectangle(axM, 'Position',[0.15 -80 0.25 0],
'FaceColor',[0 0.45 0.74], 'EdgeColor','none');
barR = rectangle(axM, 'Position',[0.60 -80 0.25 0],
'FaceColor',[0.85 0.33 0.10], 'EdgeColor','none');
pkL = yline(axM, -80, 'LineWidth', 2);
pkR = yline(axM, -80, 'LineWidth', 2);

btnPlay = uibutton(fig, 'push', 'Text','Play', 'Position',[340 110 120 40]);
btnStop = uibutton(fig, 'push', 'Text','Stop', 'Position',[340 60 120 40]);
lbl = uilabel(fig, 'Position',[490 55 560 100], 'FontSize',14, 'Text','');
```

```
sld = uislider(fig, 'Position',[340 170 720 3], 'Limits',[0 dur], 'Value',0);

% State
isPlaying = false;
dt = 0.03;
peakHoldL = -80;
peakHoldR = -80;
peakDecay_dB_per_s = 30;

tmr = timer('ExecutionMode','fixedRate', 'Period',dt, 'TimerFcn',@onTick);

% Wire callbacks
btnPlay.ButtonPushedFcn = @onPlay;
btnStop.ButtonPushedFcn = @onStop;
sld.ValueChangingFcn = @onScrub;
fig.CloseRequestFcn = @onClose;

% Initialize
updateGUI(0);

    function onPlay(~,~)
        isPlaying = true;

        t0 = sld.Value;
        if ~isempty(player)
            startSample = max(1, floor(t0*Fs)+1);
            play(player, startSample);
        end

        start(tmr);
    end

    function onStop(~,~)
        isPlaying = false;
        stop(tmr);
        if ~isempty(player)
            stop(player);
        end
    end

    function onScrub(~,evt)
        t0 = evt.Value;
        sld.Value = t0;
        updateGUI(t0);
```

```
    if ~isempty(player) && isPlaying
        stop(player);
        startSample = max(1, floor(t0*Fs)+1);
        play(player, startSample);
    end
end


function onTick(~,~)
    if hasAT && ~isempty(player) && isPlaying
        t0 = (player.CurrentSample-1)/Fs;
        if t0 >= dur
            onStop();
            t0 = dur;
        end
        sld.Value = t0;
        updateGUI(t0);
    elseif isPlaying
        t0 = min(dur, sld.Value + dt);
        sld.Value = t0;
        updateGUI(t0);
        if t0 >= dur
            onStop();
        end
    end
end

function updateGUI(t0)
    % Playheads
    ph1.Value = t0;
    ph2.Value = t0;

    % Live RMS window around current time
    winLive = max(32, round(0.050*Fs));
    iC = max(1, min(N, floor(t0*Fs)+1));
    i0 = max(1, iC - floor(winLive/2));
    i1 = min(N, i0 + winLive - 1);

    seg = x(i0:i1,:);
    rL = sqrt(mean(seg(:,1).^2));
    rR = sqrt(mean(seg(:,2).^2));

    dBL = 20*log10(rL + 1e-12);
    dBR = 20*log10(rR + 1e-12);

    % Peak hold with decay
```

```
        peakHoldL = max(peakHoldL - peakDecay_dB_per_s*dt, dBL);
        peakHoldR = max(peakHoldR - peakDecay_dB_per_s*dt, dBR);

        % Clamp display
        dBLc = max(-80, min(0, dBL));
        dBRc = max(-80, min(0, dBR));

        % Bars height in dB above -80
        barL.Position = [0.15 -80 0.25 (dBLc+80)];
        barR.Position = [0.60 -80 0.25 (dBRc+80)];
        pkL.Value = peakHoldL;
        pkR.Value = peakHoldR;

        lbl.Text = sprintf(['Time: %.3f / %.3f s\n' ...
                            'Left RMS:  %.1f dBFS   (peak %.1f)\n' ...
                            'Right RMS: %.1f dBFS   (peak %.1f)\n' ...
                            'Audio Toolbox: %s'], ...
                            t0, dur, dBL, peakHoldL, dBR, peakHoldR, ...
                            ternary(hasAT,'yes','no'));
    end

    function onClose(~,~)
        try, stop(tmr); delete(tmr); catch, end
        try, if hasAT && ~isempty(player), stop(player); end, catch, end
        delete(fig);
    end
end

function s = stripPath(p)
[~,name,ext] = fileparts(p);
s = [name ext];
end

function out = ternary(cond, a, b)
if cond, out = a; else, out = b; end
end
```